

Document number: P1050R0

Date: 2018-05-07

Reply-to: John McFarlane, fixed-point@john.mcfarlane.name

Audience: SG6, LEWG

Fractional Numeric Type

Abstract

This paper introduces a fractional number type which stores pairs of integer numerators and denominators. It avoids precision loss commonly associated with the storage of single-number quotients. It also helps express intent when initializing fixed-point types with the results of division operations.

Introduction

Rational numbers provide a headache for digital number types such as the fundamental scalars. We already have `std::ratio` -- mostly as a necessary way of expressing different static scales in `std::chrono`. There has previously been discussion of sophisticated, general purpose bounded [P0101] and unbounded [P0101] rationals. Boost has a rational type [Boost].

This paper proposes a fractional type that is compatible with the compositional approach detailed in [P0554]. As such it is designed to provide a zero-overhead abstraction over pairs of integers while also being an expressive, user-friendly, general-purpose statically-sized number type when combined with other numeric components.

The reason for proposing a fractional type at this time is because it serves a specific role in initialization of the fixed-point numbers described in [P0037].

Motivation

Class template, `fixed_point<>` from [P0037] can be initialized with integer and floating-point values. It is desirable that `fixed_point<>` values also be the quotient in a division operation. A `divide` function was previously proposed in [P0037]:

```
auto a = divide(1, 3);
```

Straight away the question arises: what should be the resolution of `a`? The solution proposed in [P0106] is to determine the number of fractional digits from the number of fractional digits in the numerator and integer digits in the denominator. In the example above, `a` has 31 fractional digits because literal, `3`, has 31 integer digits. The result has value, 0.333333333022892475128173828125.

This solution is problematic when one considers all of the ways that `1` or `3` can be represented. For example, using `elastic_integer<>` [P0828] as the inputs

```
auto b = divide(elastic_integer<1>{1}, elastic_integer<2>{3});
```

results in a `fixed_point` type with only 2 fractional digits. The resultant approximation of $1/3$ is `0.25`. This is unlikely to be the desired result in most cases.

Specifying the output type is one solution:

```
auto c = divide<fixed_point<int, -16>>(1, 3);  
auto d = divide<fixed_point<int, -16>>(elastic_integer<1>{1}, elastic_integer<2>{3});
```

Both `c` and `d` have values, `0.3333282470703125`. However, it is not entirely clear that introducing a named function specifically for this purpose is necessary, nor whether the interface is sufficiently intuitive.

By introducing a fractional type, `fractional`

```
template<class Numerator, class Denominator>
struct fractional {
    // ...

    Numerator numerator;
    Denominator denominator;
};
```

we can express the same intent using only types:

```
auto a = fixed_point{fractional{1, 3}};
auto b = fixed_point{fractional{elastic_integer<1>{1}, elastic_integer<2>{3}}};
auto c = fixed_point<int, -16>{fractional{1, 3}};
auto d = fixed_point<int, -16>{fractional{elastic_integer<1>{1}, elastic_integer<2>{3}}};
```

Synopsis

```
template<class Numerator, class Denominator>
struct fractional {
    using numerator_type = Numerator;
    using denominator_type = Denominator;

    explicit constexpr fractional(const Numerator& n, const Denominator& d);
    explicit constexpr fractional(const Numerator& n);

    template<class Scalar, _impl::enable_if_t<std::is_floating_point<Scalar>::value, int> = 0>
    explicit constexpr operator Scalar() const;

    numerator_type numerator;
    denominator_type denominator = 1;
};

template<class Numerator, class Denominator>
fractional(const Numerator& n, const Denominator&)
-> fractional<Numerator, Denominator>;

template<class Numerator>
fractional(Numerator const& n)
-> fractional<Numerator, int>;

template<class Numerator, class Denominator>
constexpr auto reduce(fractional<Numerator, Denominator> const& f);

template<class LhsNumerator, class LhsDenominator, class RhsNumerator, class RhsDenominator>
constexpr auto operator+(
    fractional<LhsNumerator, LhsDenominator> const& lhs,
    fractional<RhsNumerator, RhsDenominator> const& rhs)
-> decltype(make_fractional(
    lhs.numerator*rhs.denominator+rhs.numerator*lhs.denominator, lhs.denominator*rhs.denominator));

template<class LhsNumerator, class LhsDenominator, class RhsNumerator, class RhsDenominator>
constexpr auto operator-(
    fractional<LhsNumerator, LhsDenominator> const& lhs,
    fractional<RhsNumerator, RhsDenominator> const& rhs)
```

```

-> decltype(make_fractional(
    lhs.numerator*rhs.denominator-rhs.numerator*lhs.denominator, lhs.denominator*rhs.denominator));

template<class LhsNumerator, class LhsDenominator, class RhsNumerator, class RhsDenominator>
constexpr auto operator*(
    fractional<LhsNumerator, LhsDenominator> const& lhs,
    fractional<RhsNumerator, RhsDenominator> const& rhs)
-> decltype(make_fractional(lhs.numerator*rhs.numerator, lhs.denominator*rhs.denominator));

template<class LhsNumerator, class LhsDenominator, class RhsNumerator, class RhsDenominator>
constexpr auto operator/(
    fractional<LhsNumerator, LhsDenominator> const& lhs,
    fractional<RhsNumerator, RhsDenominator> const& rhs)
-> decltype(make_fractional(lhs.numerator*rhs.denominator, lhs.denominator*rhs.numerator));

template<class LhsNumerator, class LhsDenominator, class RhsNumerator, class RhsDenominator>
constexpr auto operator==(
    fractional<LhsNumerator, LhsDenominator> const& lhs,
    fractional<RhsNumerator, RhsDenominator> const& rhs);

template<class LhsNumerator, class LhsDenominator, class RhsNumerator, class RhsDenominator>
constexpr auto operator!=(
    fractional<LhsNumerator, LhsDenominator> const& lhs,
    fractional<RhsNumerator, RhsDenominator> const& rhs);

```

Reference Implementation

`fractional` is implemented as part of [the CNL library](#) ([header](#), [tests](#), [fixed_point integration](#)).